Geospatial Architecture Database (GAD)

AI-Assisted Development — Prompt Log

CS 4398 — Software Engineering

Group 15

Texas State University

Group Members

Oscar Puente

Brandon Stewart

Ethan Sklar

May 12, 2026

Companion to Group15SRS.html and DESIGN.md (rev. 1.13)

1. Introduction

This document catalogs the AI-assisted development prompts used to build the Geospatial Architecture Database (GAD), Group 15's CS 4398 final project. It accompanies Group15SRS.html (the Software Requirements Specification, the authoritative contract for what the system must do) and DESIGN.md (the implementation-level design document, currently at revision 1.13).

Each prompt below is paired with a description of what the AI produced and a one-line validation note explaining how the output was checked before being committed to the repository.

AI usage policy. Prompts were issued to Claude over the course of the semester. Every code output was reviewed line-by-line by a human group member before commit; nothing was merged on AI authority alone. Where the AI generated tests, those tests were run against the pre-existing pytest suite to confirm they passed or failed for the right reason when the code under test was deliberately broken.

AI generated documentation. The documentation was cross-checked against the actual code (the SRS rule for the project: code must align with the SRS, but DESIGN and README must align with the code). Drift between any two of those three artifacts was treated as a defect and corrected in the same PR that introduced it. Methodology. The team adopted a feature-branch + pull-request workflow even though we are a single team without external reviewers; every change went through a draft branch, a self-review of the diff, a green CI run (ruff + pytest on Python 3.10 and 3.12), and a squash-merge into main. The DESIGN.md revision history block was bumped on every merge that touched user-visible behavior, with cross-references to the SRS section it implemented. This audit trail is what made retroactively assembling this prompt log straightforward: every milestone in the DESIGN.md history corresponds to a prompt or small group of prompts captured below, organized by the SRS section they served. Document structure. Section 2 covers project scaffolding (the prompts that produced the initial Flask + frontend skeleton and the static reference data). Section 3 walks the SRS top-to-bottom (§3.1 through §3.7), giving the prompt(s) that produced each functional requirement. Section 4 covers infrastructure and quality — tests, CI, ORM and migrations, caching, retry policy, and the FEMA National Risk Index loader. Section 5 covers the documentation deliverables (README, DESIGN.md). Section 6 reflects on what worked, what didn't, and how the team validated AI output throughout.

2. Project Scaffolding

These prompts produced the initial repository structure, the empty Flask application, the frontend HTML/CSS/JS skeleton with the sidebar/map/dashboard layout, and the static reference data tables (state hazard profiles, IECC climate zones, building-code adoption per state, construction recommendations per hazard category). Everything below this section assumes this baseline is already in place.

Prompt 1

Initial Flask + frontend skeleton

> ***Prompt:*** *Build a Flask application that serves a single-page web app from frontend/index.html. The Flask app should live in backend/app.py with /api/* routes, and the static frontend (HTML, CSS, JS, no build step) should live in frontend/. Use a sidebar/map/dashboard three-column layout: search input + site metadata in the sidebar, an interactive map in the main panel, and a tabbed data view (Overview, Forecast, History, Alerts, Risk Score, Build Tips) below the map. Default port 5001 to avoid macOS AirPlay collisions; allow PORT env override.*
>
> **Output:** app.py with a basic route table (/, /api/health, /api/search stub), index.html with the three-pane layout, app.js with tab-switching logic, styles.css with a glass-morphism theme and CSS Grid layout. No external API calls yet.
>
> **Validation:** Manual smoke test — booted the server, confirmed the page rendered with all six tabs clickable and the layout responsive at three viewport widths (1440px, 1024px, 600px).

Prompt 2

State hazard profiles + IECC zones + building codes

> ***Prompt:*** *Create a Python module backend/db/seed_data.py with three reference dictionaries: STATE_PROFILES (51 states + DC, each with hurricane/tornado/flood/winter/heat/seismic/wildfire scores 0–10, sourced from NOAA Storm Events and FEMA hazard summaries); IECC_ZONES (per-state IECC climate zone, e.g. FL = "1/2", AK = "7/8"); BUILDING_CODES (per-state adopted IBC year and label, e.g. "FBC 2023 (IBC-based)"). Hand-curate — do not scrape.*
>
> **Output:** A 250-line seed_data.py with all three dicts populated for 50 states + DC. Sources cited in inline comments (NOAA Storm Events, FEMA National Risk Index summaries, ICC code adoption tracker).
>
> **Validation:** Wrote a test that asserts every state code appears in all three dicts — caught a missing DC entry in STATE_PROFILES on first run; fixed before commit. Test now lives at tests/test_routes.py::test_history_covers_all_us_states_and_dc.

Prompt 3

Construction recommendations per hazard category

> ***Prompt:*** *Add CONSTRUCTION_TIPS to seed_data.py: a dict keyed by the seven hazard categories (hurricane, tornado, flood, winter, heat, seismic, wildfire), each value a list of 3–6 short engineering-grade recommendations. Examples: hurricane should mention impact-rated glazing, hurricane straps, FEMA P-320 safe rooms; flood should mention finished floor elevation above the BFE, flood-vent calculation per ASCE 24; seismic should mention ASCE 7 design category, foundation anchorage. Cite FEMA / ICC / ASCE standards in the recommendation text where appropriate.*

Output: CONSTRUCTION_TIPS dict with ~30 total recommendations across the seven categories. Each tip 1–2 sentences, written in the imperative voice ("Use impact-rated glazing per…").

Validation: Cross-checked sample tips against FEMA P-320 (safe room standard) and ASCE 24 (flood design) PDFs. Three tips were rephrased after the cross-check to remove a misattribution to ASCE 7 that should have been ASCE 24.

3. SRS-Mapped Features

Each subsection below corresponds to a section of Group15SRS.html. Prompts are listed in roughly the order they were issued during development; in some cases (§3.3, §3.2) the section was implemented in two passes — an initial cut, then a closeout pass when an audit against the SRS revealed a gap. Both passes are included.

3.1 Leaflet + OpenStreetMap Integration

Prompt 4 — SRS §3.1

Leaflet map with click-to-analyze

Prompt: *Wire up a Leaflet 1.9.4 interactive map in frontend/app.js using OpenStreetMap tiles. The map should support: (1) a click handler that fires /api/weather for the clicked coordinates, (2) a marker showing the most recently analyzed location, (3) a setMapView(lat, lon) helper that pans + zooms to a coordinate. Tile attribution must credit OpenStreetMap per their license.*

Output: initMap() and setMapView() helpers in app.js. Click handler dispatches to a queryWeather(lat, lon) function. Marker auto-removed and replaced on each new analysis so only the current site is shown.

Validation: Manual test in three browsers (Chrome, Firefox, Safari): clicked five known cities and verified pin landed within visual tolerance of expected city center. Confirmed OpenStreetMap attribution renders bottom-right per their license.

Prompt 5 — SRS §3.1

Forward geocoding via Nominatim

Prompt: *Add a /api/search?q=... endpoint to backend/app.py that proxies the OpenStreetMap Nominatim API for forward geocoding. Constraints: query must be ≥3 characters (return [] otherwise to avoid hammering Nominatim); cap results to 5; only return US locations (countrycodes=us); on Nominatim outage return 503. Frontend should debounce the input by 250ms and render a suggestion list under the search box with keyboard navigation (arrow keys + Enter to select).*

Output: /api/search backend route + a renderSuggestions() function in app.js with full keyboard navigation. Empty input collapses the suggestion list.

Validation: pytest cases (5 of them) covering: query too short → [], valid query → results, Nominatim 503 → 503, empty query → [], malformed JSON response → 503. All passed.

3.2 Weather Data Retrieval and Analysis

Prompt 6 — SRS §3.2

NWS weather pull (forecast + alerts)

Prompt: *Add /api/weather?lat={f}&lon={f} that pulls current and forecast weather data from the National Weather Service API. NWS is a 3-step API: GET /points/{lat,lon} returns a forecast URL and the user's state. Then GET that forecast URL for the 7-day forecast (14 periods, day/night). Then GET /alerts/active?area={state} for any active warnings. Handle outages gracefully — return 503 with a descriptive message, never a 500. Return shape: { forecast, alerts, scores, composite, observation, state, climateZone, buildingCode }.*

Output: /api/weather endpoint with a sequential 3-call upstream chain, fan-out to NWS, normalized response shape. State is resolved from the points response or from a Nominatim reverse-geocode fallback.

Validation: pytest cases for: missing coords → 400, only-lat → 400, happy path → 200 with all expected keys, non-US (London) → 404, NWS unreachable → 503. Schema sanity assertions added so any future drift in the response shape fails loudly.

Prompt 7 — SRS §3.2

Current Conditions card on the Overview tab (closeout)

Prompt: *The /api/weather response has always carried an observation object (temperature, windSpeed, humidity, conditions sourced from the first NWS forecast period), but the Overview tab silently dropped it — those fields are fetched but never rendered. Surface them as a four-card "Current Conditions" row above the existing hazard-snapshot grid. Backend should also include temperatureUnit so the frontend doesn't have to assume Fahrenheit. If the observation is empty (NWS returned zero forecast periods), the card should suppress itself entirely rather than rendering a row of em-dashes.*

Output: A renderObservation(obs) helper in app.js; new .conditions-grid / .condition-card styles in styles.css; backend obs dict gains temperatureUnit. Card collapses when observation is empty.

Validation: 2 new pytest cases lock the field shape (5 keys: temperature, temperatureUnit, windSpeed, humidity, conditions) and the empty-periods suppression. Total suite went 79 → 81. Caught the temperatureUnit gap during peer review of the diff.

3.3 Data Export

Prompt 8 — SRS §3.3

PDF export via reportlab

Prompt: *Add POST /api/export that accepts the assembled site object as JSON and returns a styled PDF using reportlab. Sections: cover summary (location, coordinates, generated timestamp, composite risk, IECC zone, building code); hazard table (one row per category with 0–10 score and weight); construction recommendations (only categories scoring ≥3, sorted by descending score); 7-day forecast table if forecast data is in the payload; disclaimer footer citing FEMA / ICC / engineer review. Use Helvetica, slate/blue palette, US Letter portrait, 0.75-inch margins.*

Output: /api/export route with reportlab lazy-imported (so the rest of the app boots if reportlab is missing). Filename Content-Disposition: GAD_Report_YYYYMMDD.pdf.

Validation: pytest cases for: full payload → PDF magic bytes; minimal payload → valid PDF; zero active hazards → fallback "standard practices apply" message renders; empty body → doesn't 5xx.

Prompt 9 — SRS §3.3

Server-side CSV export (closeout)

Prompt: *The SRS specifies "PDF/CSV" export, but only PDF is currently rendered server-side; CSV is a hand-rolled JS Blob in the frontend whose quoting is broken on display names like "Tampa, FL". Refactor /api/export to dispatch on a ?format= query parameter (or a "format" key in the JSON body), default pdf for backward compatibility, csv as the new format. Use stdlib csv.writer for proper quoting. Mirror the PDF's section*

ordering exactly so the two artifacts are content-identical. Unknown format → 400. Frontend's downloadCSV() goes away and both formats flow through a single downloadReport(fmt) helper.

Output: _export_pdf() and _export_csv() helpers in app.py; unified downloadReport in app.js; client-side CSV builder removed. The 4 existing PDF tests continue to pass unchanged because pdf is the default.

Validation: 7 new pytest cases (CSV happy path, format-via-body, embedded-comma quoting that locks regression on the JS bug, no-alerts fallback row, unknown-format → 400, default-pdf backward-compat, minimal payload). Suite 72 → 79.

3.4 Risk + Construction Recommendations

Prompt 10 — SRS §3.4

Composite risk score

Prompt: *Add a composite_from_scores(scores) function that takes the 7-hazard score dict and returns a 0–100 weighted composite. Weights live in RISK_CATEGORIES (defined in seed_data.py). Composite = clamp(round(sum(score_i * weight_i) * 10), 0, 100). Apply per-coordinate jitter to the underlying state-level scores so two clicks at the same lat/lon produce identical results but two cities in the same state differ slightly — deterministic noise, seeded by lat+lon, range ±1.0.*

Output: composite_from_scores() and jitter() in app.py. RISK_CATEGORIES has weights summing to 1.0 (verified by a unit test).

Validation: pytest cases for: jitter is deterministic for the same coords, jitter range ∈ [−1, 1], composite always 0–100, weights sum to exactly 1.0. Caught a weight-sum drift (0.99) on first run after a tip-list refactor.

Prompt 11 — SRS §3.4

Construction recommendations rendering

Prompt: *On the Build Tips tab, render the construction recommendations from CONSTRUCTION_TIPS for any hazard scoring ≥3 in the current site's scores. Sort hazards by descending score so the most relevant guidance is at the top. Group by hazard with a heading + score badge, then a bullet list of the tips. Add a fallback ("Standard construction practices apply") when no hazard meets the threshold. Include a disclaimer at the bottom citing FEMA / ICC and engineer review.*

Output: renderRecommendations(d) function in app.js. Uses the same score ≥3 threshold and descending sort as the PDF/CSV exports so all three outputs agree on which hazards are "active."

Validation: Manual test on Tampa (high hurricane + flood), Anchorage (high seismic + winter), Phoenix (high heat). Verified the threshold + sort. Verified fallback by clicking Quito-equivalent low-risk coords.

3.5 Error Handling

Prompt 12 — SRS §3.5

Graceful degradation patterns

Prompt: *Audit the entire backend for error handling. Every external HTTP call (NWS, Nominatim, USGS, FEMA) must be wrapped in try/except*

(requests.exceptions.RequestException, ValueError, TypeError) and never let an upstream failure produce a 500. Frontend should detect online/offline (window addEventListener "offline"/"online") and show a non-blocking banner; in-flight fetches that fail while offline should retry on reconnect rather than surface a hard error to the user.

Output: Centralized try/except wrappers in /api/search, /api/weather, /api/history, /api/export. Frontend offline banner and retry queue. Toast notification system for transient errors.

Validation: Tested by killing local network mid-request and verifying the banner appeared, then restoring network and verifying the retry. Three tests added for upstream-down scenarios (Nominatim 503, NWS 503, FEMA timeout).

3.6 Analytics & Audit Log

Prompt 13 — SRS §3.6

Analyses table + best-effort audit writer

Prompt: *Add a new SQLAlchemy model Analysis (id, created_at indexed, lat, lon, state indexed nullable, composite, alert_count) plus an Alembic migration for it. Every successful /api/weather call should append one row via a best-effort _record_analysis() helper — wrap the insert in try/except so a DB failure can never break the user's analysis. SRS §4.4 says no PII; this means no IP, no user agent, no session token, no display name. Just the anonymized analysis metadata.*

Output: Analysis model, migration 0002_add_analyses_table.py, _record_analysis() helper called after the response payload is assembled. Cache hits also write a row — the user's click happened, audit log reflects it.

Validation: pytest case for: happy-path writes one row with expected fields; commit failure (mocked Session.commit raising) does not break the response. Both passed.

Prompt 14 — SRS §3.6

Read endpoints: /recent and /stats

Prompt: *Add GET /api/analyses/recent (paginated, limit ≤ 100, offset, ordered by created_at descending) and GET /api/analyses/stats (total, last24h count, byState dict, byDay dict for the trailing 14 days). Invalid query params (limit=abc, offset=xyz) should fall back to defaults instead of 4xx.*

Output: Both endpoints implemented. Filter parsing tolerant of garbage. SQL aggregations use func.count, func.strftime grouping. Results sorted deterministically.

Validation: pytest cases for: pagination correctness, garbage-param fallback, stats aggregation across 3 mocked analyses, empty-table case returns all-zero / empty-dict response.

3.7 Site Context (Elevation + FEMA Flood Zone)

Prompt 15 — SRS §3.7

USGS elevation + FEMA NFHL flood zone

Prompt: *Add two best-effort helpers to backend/app.py: get_elevation_meters(lat, lon) using USGS Elevation Point Query Service (epqs.nationalmap.gov, free, no auth), and get_flood_zone(lat, lon) using FEMA NFHL layer 28 S_FLD_HAZ_AR (hazards.fema.gov, free, no auth). Both must return None on any failure. Add elevation { meters, feet } and*

floodZone { zone, description } to /api/weather. Frontend renders both as new rows in the Site Metadata sidebar; suppress the row entirely when null. Use a 5s timeout for these (vs NWS's 8s) since values are decadal-scale and overwhelmingly cache-resident.

Output: Two new helpers + a _FLOOD_ZONE_DESCRIPTIONS map (A, AE, AH, AO, AR, A99, V, VE, X, D → human description). FEMA "no overlapping polygon" is treated as the informative answer "Zone X minimal hazard," not a failure.

Validation: 3 pytest cases: happy path with both upstreams, graceful 503 → null fallback, FEMA empty-features → Zone X informative answer. SRS amended with new §3.7 (UC1 retrieve, UC2 graceful degradation).

4. Infrastructure & Quality

These prompts produced the cross-cutting concerns that don't map to a specific SRS functional requirement but contribute to SRS Part 4 non-functional requirements: §4.1 Reliability (test suite, retry adapter, TTL cache), §4.2 Robustness (graceful degradation, isolation), §4.3 Maintainability (CI, ORM/migrations, living docs).

Prompt 16 — SRS §4.1, §4.3

pytest scaffolding + first 36 tests

Prompt: *Set up tests/ with a conftest.py that imports the Flask app, provides a test client, and stubs external HTTP via pytest-mock. Use an in-memory SQLite DB (GAD_DATABASE_URL=sqlite:///:memory: set before app import). Add an autouse fixture that clears the weather TTL cache between tests. Write the first 36 tests covering /api/search, /api/weather, /api/history, /api/export, /api/health, plus utility tests for normalize_state, jitter, composite_from_scores. Use named fixtures (nws_point_payload, nominatim_search_payload, etc.) so the route tests are terse.*

Output: tests/ directory with conftest.py, test_routes.py, test_utils.py, test_export.py. _FakeResponse helper. All 36 tests passing on first full run.

Validation: Suite ran in 0.4s on the dev machine. Rendered ruff clean. Committed as the baseline.

Prompt 17 — SRS §4.3

GitHub Actions CI

Prompt: *Create .github/workflows/ci.yml that runs on every push to main and every pull request. Two-version matrix (Python 3.10 and 3.12). Steps: checkout, setup-python, pip install -r backend/requirements-dev.txt (transitively pulls in runtime deps), ruff check ., pytest -v. Cache pip wheels keyed on the requirements files. Branch protection on main should require both matrix legs to pass before merge.*

Output: ci.yml with ~30 lines, matrix build, wheel cache. Branch protection configured in the GitHub UI.

Validation: Triggered the workflow with a deliberately-broken commit (a syntax error in app.py); CI failed as expected and blocked the PR. Reverted, CI went green.

Prompt 18 — SRS §4.3

SQLAlchemy 2.0 ORM migration

Prompt: *Migrate the static reference dicts (STATE_PROFILES, IECC_ZONES, BUILDING_CODES, HISTORICAL_EVENTS, DECADAL_TRENDS, RISK_CATEGORIES, CONSTRUCTION_TIPS) into a SQLite database accessed via the SQLAlchemy 2.0 ORM. Schema in backend/db/models.py (typed declarative). Idempotent seed loader in backend/db/seed.py that reads the canonical Python representation in seed_data.py and inserts rows on first boot. init_db() called on app import. Tests use sqlite:///:memory: with a StaticPool.*

Output: backend/db/ package with __init__.py, models.py, seed_data.py, seed.py. RISK_CATEGORIES and CONSTRUCTION_TIPS re-exported from seed_data so the PDF path and tests still use them as Python dicts.

Validation: All 52 existing tests still passing; no new test failures. Migration verified by deleting backend/gad.db and confirming the app rebuilt the schema + seeded reference data on next boot.

Prompt 19 — SRS §4.3

Alembic schema migrations

Prompt: *Move the schema-versioning responsibility from Base.metadata.create_all() to Alembic. Add alembic.ini and alembic/env.py at the repo root. env.py should prefer a connection passed via Config.attributes["connection"] over building its own engine — critical so in-memory SQLite tests don't end up with two unrelated :memory: databases. Generate migration 0001_initial_schema covering all five tables + indexes. init_db() now calls alembic.command.upgrade(cfg, "head") instead of create_all.*

Output: alembic.ini, alembic/env.py, alembic/script.py.mako, alembic/versions/0001_initial_schema.py. CI verifies a fresh DB picks up the migrations correctly.

Validation: Tested both code paths: in-memory test DB and on-disk gad.db. Both end at the same schema version. Tests still passing on both Python 3.10 and 3.12.

Prompt 20 — SRS §4.1

TTL cache + retry adapter

Prompt: *Add a thread-safe TTLCache (OrderedDict + threading.Lock, LRU eviction, hit/miss counters) at backend/cache.py. Cache /api/weather responses for 5 minutes keyed on (round(lat,3), round(lon,3)) so nearby clicks share an entry without crossing state lines. Cache hits still write an Analysis row — user's click happened, audit log reflects it. Separately, mount a urllib3 Retry adapter on a shared requests.Session at backend/http_session.py: total=2, backoff_factor=0.5, status_forcelist=(502,503,504). Migrate every requests.get to http.get.*

Output: backend/cache.py + backend/http_session.py. /api/cache/stats endpoint surfaces the hit-rate. Conftest autouse fixture clears the cache between tests so test ordering is irrelevant.

Validation: 6 new pytest cases (cache miss-then-hit, nearby-coords-share-cell, TTL expiry via mocked time.monotonic, /api/cache/stats shape + counters, retry adapter policy assertion). Suite 60 → 66.

Prompt 21 — SRS §3.4

FEMA NRI county data integration

Prompt: *Add a county-level NRICounty model + migration. Loader at backend/db/nri_loader.py parses FEMA's NRI_Table_Counties.csv schema (HRCN_RISKS, TRND_RISKS, CFLD_RISKS, IFLD_RISKS, etc.). Normalize 0–100 percentile to our 0–10 internal scale. Map FEMA's 18 hazard categories to our 7 (flood = max(CFLD, IFLD); winter = max(WNTW, ISTM, CWAV); etc.). Idempotent two-pass loader: parse file fully into memory before truncating the table, so an HTML stub accidentally saved as nri_counties.csv (curl without -L through a redirect) doesn't destroy existing seed data. /api/weather prefers county-level data when the NWS county zone id resolves; falls back to state-level otherwise.*

Output: NRICounty model, migration 0003_add_nri_counties_table.py, nri_loader.py with parse_nri_row() (pure function, exposed for tests) + load_nri_counties()

(DB-touching). 15-county sample committed at backend/data/nri_sample.csv; full FEMA download is gitignored.

Validation: 3 new pytest cases (county hit path, state-level fallback, loader does-not-truncate-on-invalid-input regression test). Suite 66 → 69. The truncation-protection test specifically reproduces a real failure mode observed during integration.

5. Documentation

Prompt 22

README structure

Prompt: *Write README.md as the top-level project document. Sections: project description (one paragraph elevator pitch), what it does (numbered list of user-visible capabilities), quick start (clone + venv + pip install + run, with separate macOS/Linux and Windows blocks), project structure (annotated directory tree), database section explaining the SQLAlchemy ORM and Alembic migrations, FEMA NRI section explaining the optional production data download, API endpoints (table with method, path, purpose, SRS reference), data sources, architecture diagram, testing instructions, contributors, license. Match Group15SRS.html as the source of truth for any contract claims.*

Output: ~240-line README.md with all the above sections. Includes a table-of-contents anchor list at the top and a "Last reviewed" date at the bottom that the team agreed to bump on every doc-touching PR.

Validation: Cross-checked every API endpoint claim against the actual routes in app.py. Bumped the "Last reviewed" date convention into the team's living-docs rule (see Section 6).

Prompt 23

DESIGN.md

Prompt: *Write DESIGN.md as the implementation-level design document. Sections: purpose & scope (and what differs from the SRS — SRS says what, DESIGN says how); references (NWS, OSM, IECC, IBC, ASCE, FEMA P-320); system overview; high-level architecture (Mermaid flowchart); component breakdown (frontend / backend / data layer / dependencies); data flow (sequence diagram for the canonical "user clicks the map"); API design (per-endpoint contract); data model (Mermaid ER diagram); algorithms (alert info-URL mapping, risk scoring); external dependencies (runtime, dev, frontend, network); error-handling strategy; non-functional compliance; SRS → implementation traceability matrix; deployment & operations; future work / known limitations; revision history.*

Output: ~640-line DESIGN.md, version 1.0 at first commit, currently at v1.13. Revision history block bumped on every PR that touches user-visible behavior.

Validation: Used the traceability matrix as a self-audit during every PR — if a feature didn't land in the matrix it usually meant a missing implementation file or a missing test. Caught two such gaps over the semester.

6. Reflection

What worked well. The biggest win was the discipline of having three artifacts. The SRS, the implementation, and DESIGN.md stay in sync, with the SRS as the contract. Every PR that touched user-visible behavior also bumped the DESIGN revision history, and every PR that introduced a new functional capability either implemented an existing SRS section or amended the SRS in the same PR. This made AI-assisted code generation much safer than it would have been otherwise: the AI was constrained by the existing SRS section to produce code that fit the contract, and any drift was caught at PR review by comparing the diff to the relevant SRS section. The traceability matrix in DESIGN §13 was the single most useful artifact for catching gaps; it failed in two cases over the semester (a missing test, a missing endpoint), and both were caught before merge.

What didn't work as cleanly. AI-generated tests were occasionally too optimistic — they tested the happy path well but missed the edge cases that real users hit (the FEMA flood-zone "no overlapping polygon" case, for example, was not in the AI's first test draft; we added it after manual exploration of the API). The fix was a personal habit: every time the AI produced tests for a new feature, the human reviewer would deliberately try to break the production code in three or four ways and check whether the tests caught each break. If they didn't, the missing assertions went into the same PR. We also discovered that AI-generated documentation needs cross-checking against the actual code, not just trusted on AI authority — the API endpoint table in the README was wrong twice over the semester (a stale field name, a missing endpoint), caught only by reading the actual route definitions.

How outputs were validated. Every code change went through four gates: (1) the human reviewer read the diff line-by-line, (2) ruff check . ran clean (lint and import ordering), (3) pytest -v ran fully green (suite grew from 36 to 84 over the semester), (4) the developer manually smoke-tested the change against a fresh app boot, with a specific script of inputs (Tampa, Anchorage, Phoenix, Honolulu, plus one deliberately bad input). For backend changes, the smoke test included a curl against the affected endpoint to confirm the response shape matched the documented contract in DESIGN §7. For frontend changes, the smoke test included visual confirmation in three browsers and a print-preview check.

Two things that we would do differently, would be more thorough with the initial SRS, to allow for expansion. Adding features that weren't originally in the SRS led to complications,

AI assistance was a force multiplier on this project but only when paired with the existing software-engineering disciplines this course emphasizes — a written SRS as the contract, a feature-branch + PR workflow, a CI-gated test suite, a living design document. Without those, AI output would have been hard to validate; with them, the AI was effectively a fast and energetic teammate who needed all of its work reviewed before it landed. The final system is, in our judgment, a more thoroughly documented, more thoroughly tested, and more SRS-compliant codebase than it would have been if the team had built it without AI assistance, but only because we built the AI use into the existing process rather than letting it bypass the process.